Foldable & Traverse

Foldable/Traverse · Fold scores; traverse validated IDs · Build with Waqar

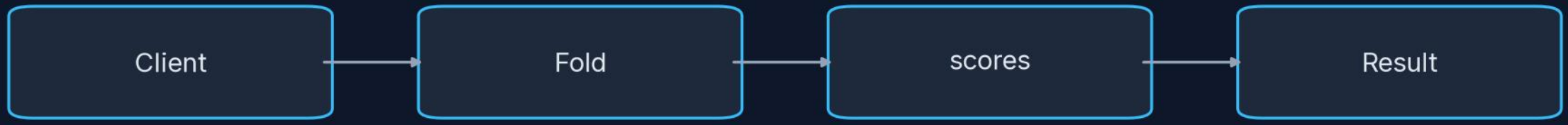
Today's Hook

- Problem: Fold scores; traverse validated IDs
- Pattern: Foldable/Traverse — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Fold scores; traverse validated IDs
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Foldable/Traverse: Fold scores; traverse validated IDs

Real-World Scenario

- Running example: Fold scores; traverse validated IDs
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.foldable;

import com.buildwithwaqar.patterns.functional.foldable.Foldable;
import com.buildwithwaqar.patterns.functional.monoid.Monoids;
import java.util.List;
/** Problem: generic fold of scores with a monoid. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Foldable problem: score totals ===");
    System.out.println(Foldable.fold(List.of(10,20,30), Monoids.INT_SUM));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.foldable;

import com.buildwithwaqar.patterns.functional.monoid.Monoid;
import com.buildwithwaqar.patterns.functional.typeclass.Applicative;
import com.buildwithwaqar.patterns.functional.typeclass.Kind;

import java.util.ArrayList;
import java.util.List;
import java.util.function.BiFunction;
import java.util.function.Function;

/** Foldable + Traverse for List — educational encodings. */
public final class Foldable {
    private Foldable() {}

    public static <A, B> B foldLeft(List<A> xs, B z, BiFunction<B, A, B> f) {
        B acc = z;
        for (A x : xs) acc = f.apply(acc, x);
        return acc;
    }

    public static <A> A fold(List<A> xs, Monoid<A> m) {
        return foldLeft(xs, m.empty(), m::combine);
    }
```

Solution (2)

```
    public static <F, A, B> Kind<F, List<B>> traverse(Applicative<F> A, List<A> xs, Function<A, Kind<F, B>> f) {
        Kind<F, List<B>> acc = A.pure(new ArrayList<>());
        for (A x : xs) {
            Kind<F, B> fb = f.apply(x);
            acc = A.ap(A.map(acc, list -> (b -> { list.add(b); return list; })), fb);
        }
        return acc;
    }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.foldable;

import com.buildwithwaqar.patterns.functional.foldable.Foldable;
import com.buildwithwaqar.patterns.functional.monoid.Monoids;
import java.util.List;
/** Problem: generic fold of scores with a monoid. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Foldable problem: score totals ===");
    System.out.println(Foldable.fold(List.of(10,20,30), Monoids.INT_SUM));
  }
}
```

Walkthrough Output

```
=== Foldable problem: score totals ===
60
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Foldable/Traverse.

Related Patterns

- Contrast Foldable/Traverse with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Foldable/Traverse → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.